

EXP7:WIRESHARK STATISTICS AND GRAPH

RAVI BALA M.V

Register Number: 24BCE1419

Faculty Name: Subbulakshmi

Lab Slot: L55 + L56

Course Name: Computer Networks Lab

Course Code: BCSE308L

AIM

To Explore the statistics and Graphs menu of wireshark

SOFTWARE USED

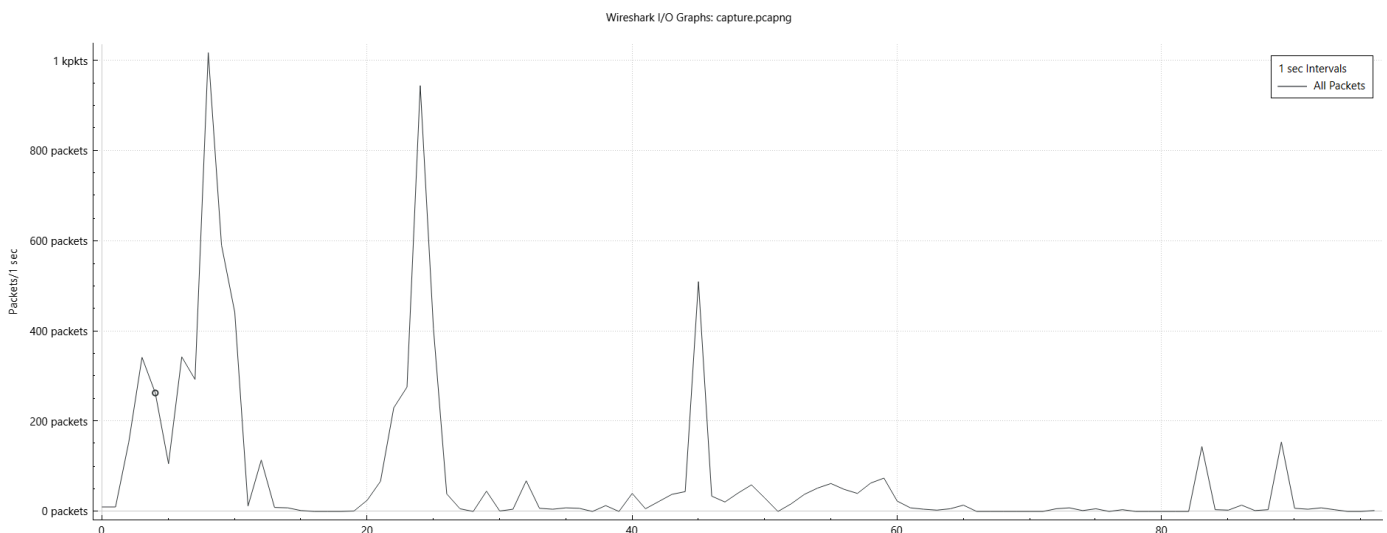
Tool: Wireshark,spyder,anaconda,scapy

PROCEDURE:

- Capture live network traffic and save capture files.
- Go to statistics and select the IO Graphs menu and create any 5 graphs to display the visual details of the packet capture.
 - o Save the graphs
 - o Write 5 lines of inferences for every graph
 - o Create the same 5 graphs using scapy library and python program
- Go to protocol hierarchy and save this detail as a *.csv file and print the following using scapy program
 - o Total number of packets
 - o Size of data
 - o Size of headers
- Go to the conversations menu and save this file as a *.csv file and print the following using a scapy program
 - o The unique address pairs which communicates maximum bytes of data
 - o The average inter – packet time difference between every pair of packets.
 - o Total number of packets transferred between every pair and every protocol

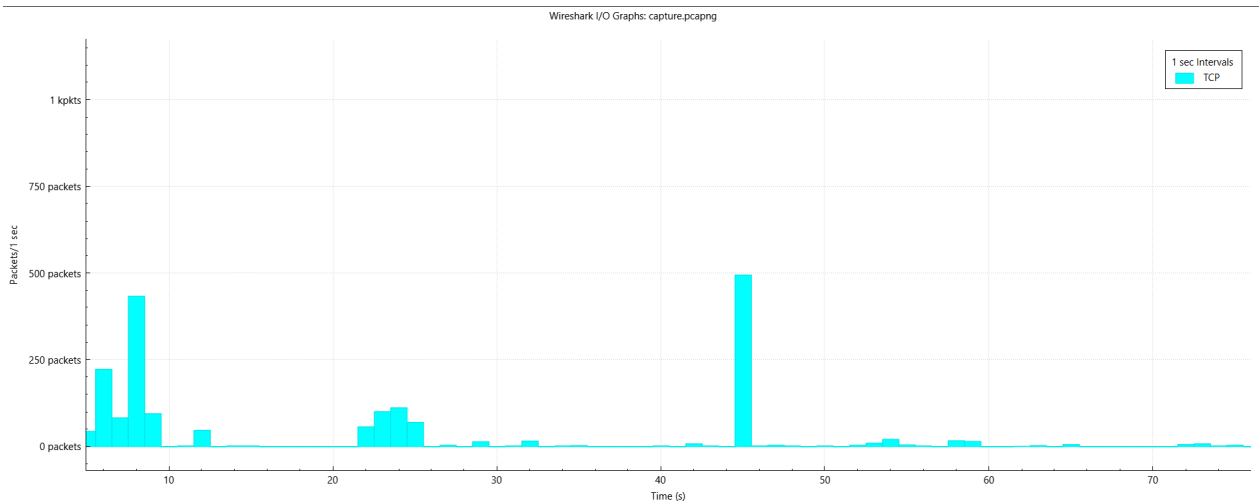
OBSERVATION(graph):

1.



1. The highest overall traffic spike occurs around 7–10 seconds.
2. Another major spike appears near 23–25 seconds.
3. A moderate spike is visible near 45 seconds.
4. Traffic reduces significantly after 60 seconds.
5. The overall pattern suggests browsing followed by idle intervals.

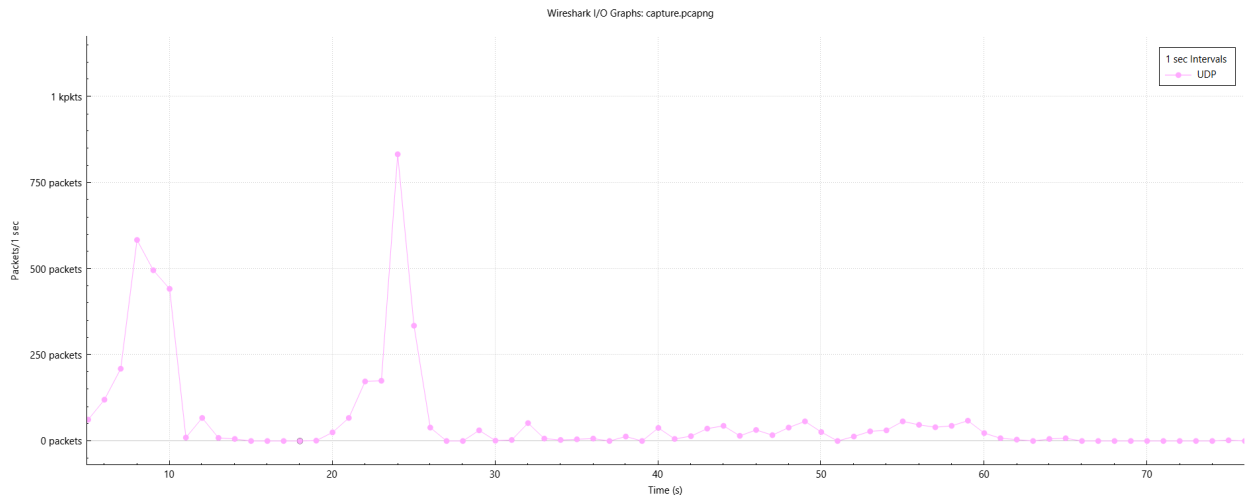
2.TCP



TCP traffic shows significant spikes around 5–10 seconds and 22–25 seconds, indicating active data exchange.

1. A major peak near 45 seconds suggests a large data transfer or web content loading.
2. After peak periods, traffic stabilizes, indicating normal browsing behavior.
3. TCP dominates traffic during high activity intervals.
4. The pattern shows reliable, connection-oriented communication typical of web browsing.

3.UDP



1. UDP traffic shows sharp bursts, especially near 8–10 seconds and 23–25 seconds.

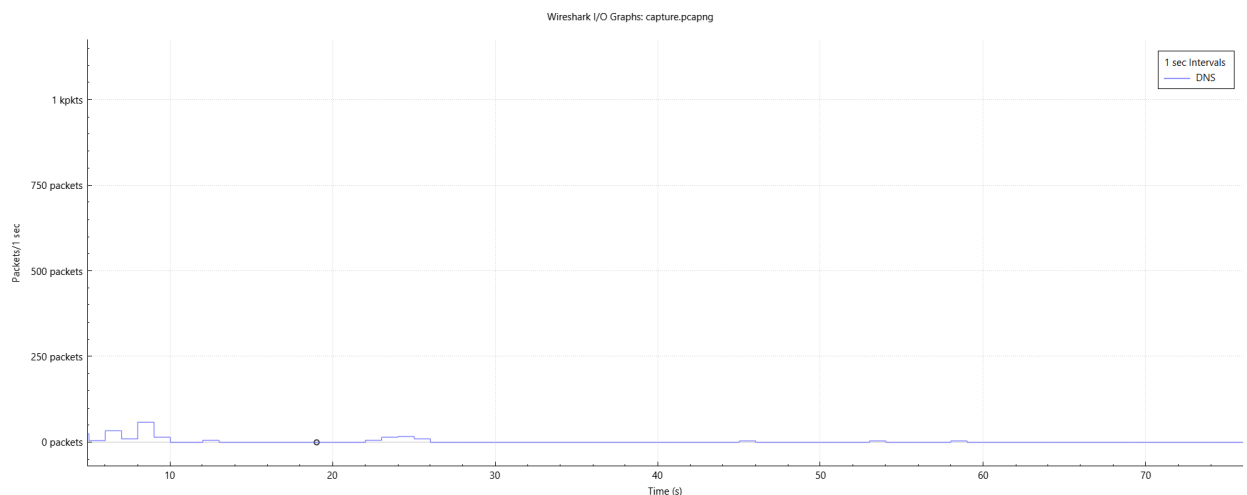
2. The highest UDP spike is around 23–24 seconds, possibly due to DNS or streaming activity.

3. UDP traffic is more irregular compared to TCP.

4. UDP packets appear in short bursts rather than continuous flow.

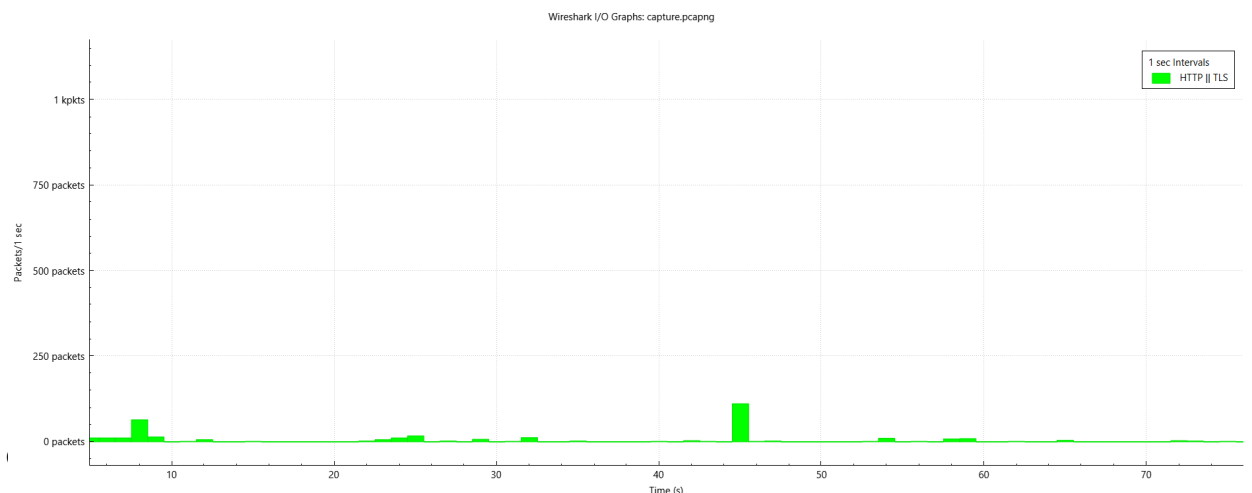
5. Overall UDP traffic volume is lower than TCP during most time intervals.

4. DNS



1. DNS traffic appears mostly at the beginning (5–10 seconds) of the capture.
2. DNS activity is minimal compared to TCP and UDP.
3. Small periodic spikes indicate domain name resolution events.
4. DNS packets are short and infrequent.
5. After the initial browsing phase, DNS traffic decreases significantly.

5. HTTPS || TLS



2. Activity aligns with TCP spikes, confirming web-based communication.
3. Packet count increases when pages are loaded.
4. Traffic remains low during idle periods.

CODE AND RESPECTIVE OUTPUT:

THIS CODE IS FOR THE PROTOCOL HIREARCHY CALCULATION:

```

from scapy.all import rdpcap, Raw

packets = rdpcap("capture.pcap")

print("==== PROTOCOL ANALYSIS ====")
print("Total number of packets:", len(packets))

total_size = sum(len(pkt) for pkt in packets)

data_size = 0
header_size = 0

for pkt in packets:
    if pkt.haslayer(Raw):
        data_size += len(pkt[Raw])
        header_size += len(pkt) - len(pkt[Raw])
    else:
        header_size += len(pkt)

print("Total size of data:", data_size, "bytes")
print("Total size of headers:", header_size, "bytes")
print("Total size of capture:", total_size, "bytes")

```

OUTPUT:

Name ▲	Type	Size	Value
data_size	int	1	3446585
header_size	int	1	481029
packets	plist.PacketList	7546	PacketList object of scapy.plist module
pkt	layers.12.Ether	54	Ether object of scapy.layers.12 module
total_size	int	1	3927614

THIS CODE IS FOR CONVERSATION CALCULATIONS:

```

import pandas as pd
print("\n==== CONVERSATION ANALYSIS ====")
df = pd.read_csv(r"C:\Desktop\Analysis\ipv4_conversation.txt")
df.columns = df.columns.str.strip()
print("\nTotal Conversations:", len(df))
print("\nColumns detected:", df.columns.tolist())

if 'Bytes' in df.columns:
    max_pair = df.loc[df['Bytes'].idxmax()]

```

```

print("\nPair with Maximum Data Transfer:")
print("Address A:", max_pair.get('Address A'))
print("Address B:", max_pair.get('Address B'))
print("Total Bytes:", max_pair.get('Bytes'))
else:
    print("\n'Bytes' column not found. Check column name.")

if 'Duration' in df.columns:
    avg_duration = df['Duration'].mean()
    print("\nAverage Conversation Duration:", avg_duration)
else:
    print("\n'Duration' column not found.")

if 'Packets' in df.columns:
    print("\nPackets between each pair:")
    for i in range(len(df)):
        print(df.loc[i].get('Address A'), "<->",
              df.loc[i].get('Address B'),
              "Packets:", df.loc[i].get('Packets'))
else:
    print("\n'Packets' column not found.")

```

OUTPUT:

Name	Type	Size	Value
avg_duration	float64	1	31.44742925
data_size	int	1	3446585
df	DataFrame	(8, 13)	Column names: Address A, Address B, Packets, Bytes, Stream ID, Packets ...
header_size	int	1	481029
i	int	1	7
max_pair	Series	(13,)	Series object of pandas.core.series module
packets	plist.PacketList	7546	PacketList object of scapy.plist module
pkt	layers.l2.Ether	54	Ether object of scapy.layers.l2 module
total_size	int	1	3927614

OUTPUT IN TERMINAL TEXT:

===== CONVERSATION ANALYSIS =====

Total Conversations: 8

Columns detected: ['Address A', 'Address B', 'Packets', 'Bytes', 'Stream ID', 'Packets A → B', 'Bytes A → B', 'Packets B → A', 'Bytes B → A', 'Rel Start', 'Duration', 'Bits/s A → B', 'Bits/s B → A']

Pair with Maximum Data Transfer:

Address A: 10.172.239.47

Address B: 10.172.239.180

Total Bytes: 107198

Average Conversation Duration: 31.44742925

Packets between each pair:

3.111.241.224 <-> 10.172.239.47 Packets: 34

10.172.239.47 <-> 10.172.239.180 Packets: 1626

10.172.239.47 <-> 13.107.42.254 Packets: 4

10.172.239.47 <-> 13.159.9.15 Packets: 28

10.172.239.47 <-> 35.210.188.173 Packets: 8

10.172.239.47 <-> 72.25.64.2 Packets: 25

10.172.239.47 <-> 130.211.36.94 Packets: 38

10.172.239.47 <-> 204.79.197.222 Packets: 4